Reconstructing constants of motion from time series data

Jonas Katona

November 15, 2022

1 Introduction

2 Initial system

Suppose that we have some observed time series data $\xi := \{x_0, x_1, \ldots, x_{n-1}\} \in \mathbb{R}^d$ belonging to a trajectory of the autonomous ODE

$$\left\{ \frac{d}{dt} x(t) = f\left(x(t)\right) \right. \tag{1}$$

at times $t_0 < t_1 < \cdots < t_{n-1}$, respectively, where $f : O \to \mathbb{R}^d$ is a C^2 vector field defined on some open subset $O \subseteq \mathbb{R}^d$ containing ξ. Furthermore, suppose that our ODE in (1), and hence, ξ as well, both exhibit n_0 differentiable integrals of motion $Q_1, \ldots, Q_{n_0} : [t_0, t_{n-1}] \times O \to \mathbb{R}$ a priori such that $\frac{d}{dt} Q_j\left(t, x(t)\right) = \partial_t Q + f\left(x(t)\right) \cdot \nabla Q_j\left(t, x(t)\right) = 0$ and $j = 1, \ldots, n_0$ for all $t \in [t_0, t_{n-1}]$, i.e., $Q_j\left(t, x(t)\right) = C_j$ for constants $C_1, \ldots, C_{n_0} \in \mathbb{R}$ for all $t \in [t_0, t_{n-1}]$. We desire to explore methods which reconstruct $Q_1, \ldots, Q_{n_0}$ given the observations above.

3 Method 1A: General square-of-sum seminorms

Let $\Xi := \{(t_0, x_0), (t_1, x_1), \ldots, (t_{n-1}, x_{n-1})\}$. Without loss of generality, suppose that Ξ belongs to only one initial condition and that (1) exhibits only one integral of motion, Q, such that $Q(t, x) = C \in \mathbb{R}$ for all $(t, x) \in \Xi$. (We can readily generalize our results for multiple initial conditions by considering the sum of seminorms, each of which is defined on a trajectory corresponding to a different initial condition than the rest.) Furthermore, suppose that we expand Q in a finite basis of $N \in \mathbb{N}$ functions $g = (g_1, \ldots, g_N)$ such that $Q \approx \sum_{j=1}^{N} a_{0,j} g_j$ for a best-fit vector of coefficients $a_0 = (a_{0,1}, \ldots, a_{0,N})$ which provides a best-fit approximation for Q in the vector space $\mathcal{G} = \operatorname{Span} g$. There are two seminorms we can use to measure "how well" a given linear combination of elements from $\mathcal{G}$ approximates Q on the time series ξ given. These can be used to derive two methods for computing a_0, of which neither requires us to know f, the vector field that defines (1).

Intuitively speaking, as we will see in a bit, the variance seminorm measures how much a candidate constant of motion deviates from its average over all such points in the time series, while the deviation seminorms does the same across every other point in the time series either forward or backwards in time. However, if we wanted to weight the deviations of a candidate constant of motion differently at each point in the time series, then we could

consider the general functional

$$V(F) = \frac{1}{2}\sum_{k=0}^{n-1}\left[\sum_{l=0}^{n-1} b_{kl}\left(F(t_k, x_k) - F(t_l, x_l)\right)\right]^2 \tag{2}$$

defined from $C(\Xi, \mathbb{R})$ to $\mathbb{R}_{\geq 0}$, where $b_{kl} \in \mathbb{R}$ for all $(k, l) \in \{0, 1, \ldots, n-1\}^2$. We desire to minimize (9) over $\mathcal{G}$. Since every element in $\mathcal{G}$ can be written as a linear combination of $\mathcal{G} := \{g_n\}_{n=1}^N$, we can now rethink of (2) as being a function of a when restricted to $\mathcal{G}$, i.e.,

$$\begin{aligned}
\widetilde{V}(a) := V(F)|_{\mathcal{G}} &= \frac{1}{2}\sum_{k=0}^{n-1}\left[\sum_{l=0}^{n-1} b_{kl}\left(\sum_{j=1}^{N} a_j g_j(t_k, x_k) - \sum_{j=1}^{N} a_j g_j(t_l, x_l)\right)\right]^2 \\
&= \frac{1}{2}\sum_{k=0}^{n-1}\left[\sum_{j=1}^{N} a_j \sum_{l=0}^{n-1} b_{kl}\left(g_j(t_k, x_k) - g_j(t_l, x_l)\right)\right]^2 \\
&= \frac{1}{2}\sum_{k=0}^{n-1}\left[\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)^T a\right]^2.
\end{aligned} \tag{3}$$

Since every seminorm is a sublinear function, every seminorm on a vector space is convex. Consequently, every critical point of (3) is a local minimum, and hence, any best-fit choice of coefficients a_0 should satisfy $\left.\frac{d\widetilde{V}(a)}{da}\right|_{a=a_0} = 0$. By vector chain rule and product rule,

$$\begin{aligned}
\frac{d\widetilde{V}(a)}{da} &= \frac{1}{2}\sum_{k=0}^{n-1}\frac{d}{da}\left[\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)^T a\right]^2 \\
&= \sum_{k=0}^{n-1}\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)^T a\right)\frac{d}{da}\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)^T a\right) \\
&= \sum_{k=0}^{n-1}\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)^T a\right)\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)^T\right) \\
&= a^T\sum_{k=0}^{n-1}\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)\right)^T\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)\right),
\end{aligned} \tag{4}$$

such that a_0 satisfies

$$a_0^T\sum_{k=0}^{n-1}\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)\right)^T\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)\right) = \mathbf{0}^T, \tag{5}$$

where $\mathbf{0}_n = \left(\overbrace{0, \ldots, 0}^{n \text{ times}}\right)$, or after taking transposes of both sides of (5),

$$\sum_{k=0}^{n-1}\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)\right)\left(\sum_{l=0}^{n-1} b_{kl}\left(g(t_k, x_k) - g(t_l, x_l)\right)\right)^T a_0 = \mathbf{0}. \tag{6}$$

Finally, it is known from linear algebra that the transpose of a matrix times itself is equal to the sum of outer products created by its rows. By this property, we can rewrite (6) as

the following:

$$M^T M a_0 = \mathbf{0}_n, \quad M = \begin{pmatrix} \leftarrow & \sum_{l=0}^{n-1} b_{0l} \left(g\left(t_0, x_0\right) - g\left(t_l, x_l\right) \right) & \rightarrow \\ \leftarrow & \sum_{l=0}^{n-1} b_{1l} \left(g\left(t_1, x_1\right) - g\left(t_l, x_l\right) \right) & \rightarrow \\ & \vdots & \\ \leftarrow & \sum_{l=0}^{n-1} b_{(n-1)l} \left(g\left(t_{n-1}, x_{n-1}\right) - g\left(t_l, x_l\right) \right) & \rightarrow \end{pmatrix}. \tag{7}$$

TODO: Add a note about cases when the matrix $[B]_{kl} = b_{kl}$ is singular because the two deviation seminorms actually correspond to this case. To compute M, it might be advantageous to express the coefficients b_{kl} as a matrix, in which case each row of M can be expressed as the vector which results from the matrix-vector multiplication between the $N \times n$ matrix with entries $g\left(t_k, x_k\right) - g\left(t_l, x_l\right)$ and the $n \times 1$ vector b_{kl}.

(7) defines a matrix equation which must be solved for a_0. Clearly, $a_0 = 0$ is a trivial solution for (7), but since we are trying to look for a nontrivial integral of motion, we want to find solutions where $a_0 \neq 0$. Such solutions should lie in $\ker M^T M = \ker M$. Ideally, there would exist an $a_0 \neq 0$ which solves (7) exactly, but if our time series ξ exhibits some measurement error due to noise, or if Q does not actually lie in $\mathcal{G}$ and we are merely using g to approximate Q, then we might need to find an approximate non-trivial null space for M, since M may only be near-singular due to noise or Ξ may not be large enough to reconstruct Q exactly. In fact, M could be a non-singular matrix (albeit, likely near-singular if such deviations or approximations are small) or (7) could define an overdetermined system when $n \gg N$, as might be preferable if our time series data is large or because we risk overfitting, and which would happen if well over N vectors from $\{g\left(t_0, x_0\right) - \overline{g}, \ldots, g\left(t_{n-1}, x_{n-1}\right) - \overline{g}\}$ are linearly independent.

To approximate ker M, and hence a_0, we appeal to the Eckart–Young–Mirsky theorem. Suppose that the singular value decomposition of M is $U \Sigma V^T$, where U and V are orthogonal matrices with column vectors $u_1, \ldots, u_n$ and $v_1, \ldots, v_N$, respectively, and Σ is a diagonal matrix of the same size as M, i.e., $\Sigma \in \mathbb{R}^{n \times N}$, with entries $\sigma_1 \geq \cdots \geq \sigma_N$ that correspond to the singular values of M. Since U and V are orthogonal, we see that

$$M^T M = \left(U \Sigma V^T\right)^T \left(U \Sigma V^T\right) = V \Sigma U^T U \Sigma V^T = V \Sigma I \Sigma V^T = V \Sigma^2 V^T. \tag{8}$$

Hence, the best rank k approximation to $M^T M$ with the error measured in any unitarily invariant norm $||\cdot||$ is $\sum_{j=1}^{k} \sigma_j^2 v_j v_j^T$, such that $\left|\left|M^T M v_j\right|\right| = \sigma_j^2$ for $j = 1, \ldots, N$. Thus, if we want to test candidate solutions for a_0, we can set some a priori error tolerance $0 \leq \tau \ll 1$ and choose $1 \leq k \leq N$ such that $\sigma_m^2 \leq \tau$ for all $k \leq m \leq N$. Finally, we let $a_0 = \operatorname{argmin}_{k \leq m \leq N} \widetilde{V}\left(v_m\right)$, to which end we can also consider the arguments for the second, third, etc. minima of $\widetilde{V}\left(\{v_m\}_{m=k}^N\right)$ to compute other possible integrals of motion.

However, since the right singular vectors corresponding to the smallest singular values tend to be very sensitive to small errors in $M^T M$, and even more so when $M^T M$ is near-singular, it is usually unwise to compute $\{v_k, \ldots, v_N\}$ directly. Instead, we will have better accuracy computing the subspace spanned by $\{v_k, \ldots, v_N\}$, which is in fact equal to the null space of the rank $N - k$ approximation for $M^T M$: $\ker\left(\sum_{j=1}^{N-k} \sigma_j^2 v_j v_j^T\right)$. Hence, any approximate constant of motion lies in $\ker\left(\sum_{j=1}^{N-k} \sigma_j^2 v_j v_j^T\right) = \operatorname{span}\{v_k, \ldots, v_N\}$ because a linear combination of constants of motion is still a constant of motion, and if we want to express this null space using a "nicer" basis for notational ease, we can use a sparse basis algorithm to ensure that our basis elements for $\ker\left(\sum_{j=1}^{N-k} \sigma_j^2 v_j v_j^T\right)$ has the least amount of zeros; see Brualdi et al. (1995), Qu et al. (2015), Khorramizadeh and Mahdavi-Amiri (2013), or Coleman and Pothen (1984) for details.

TODO: Conduct error analysis: How does our solution for a_0 vary depending off of errors in M? See, e.g., https://arxiv.org/pdf/1801.10512.pdf and https://arxiv.org/pdf/1601.03678.pdf.

We now consider special choices of coefficients b_{kl} which lead to seminorms that are particularly intuitive.

3.1 The variance seminorm

When $b_{kl} = 1/n$, we have the functional

$$V(F) = \frac{1}{2}\sum_{k=0}^{n-1}\left(F(t_k, x_k) - \frac{1}{n}\sum_{l=0}^{n-1} F(t_l, x_l)\right)^2. \tag{9}$$

It is not hard to show that (9) defines a seminorm on $C(\Xi, \mathbb{R})$; (9) is not a norm because every constant function on $C(\Xi, \mathbb{R})$ will be a zero for (9). We call V the *variance* seminorm because (9) looks almost identical to the expression for the variance of a discrete random variable which takes on every value in Ξ with equal probability. Consequently, when V is zero for Q, then Q does not deviate at all from its average for every point in Ξ, i.e., Q is constant on Ξ. Going further with this analogy, for shorthand, we will write $\overline{F} = \frac{1}{n}\sum_{l=0}^{n-1} F(t_l, x_l)$ from now on. Then, finding the minimum of (9) over $\mathcal{G}$ is equivalent with solving the following problem for a_0:

$$M^T M a_0 = \mathbf{0}_N, \quad M = \begin{pmatrix} \leftarrow & g(t_0, x_0) - \overline{g} & \rightarrow \\ & \vdots & \\ \leftarrow & g(t_{n-1}, x_{n-1}) - \overline{g} & \rightarrow \end{pmatrix}. \tag{10}$$

3.2 The backward deviation seminorm

We now look at the functional

$$V(F) = \frac{1}{2}\sum_{k=1}^{n-1}\left[\sum_{0\le l<k}\left(F(t_l, x_l) - F(t_k, x_k)\right)\right]^2 \tag{11}$$

which corresponds to when $b_{kl} = I_{l<k}$. Again, one can show that D defines a seminorm on $C(\Xi, \mathbb{R})$, which we call the *deviation* seminorm because (11) consists of the squared pairwise differences between values of F on ξ, where the outer sum is taken for $l < k$ to avoid double counting. Hence, D essentially measures how much its argument deviates in value from itself over ξ; when $D = 0$, that means that its argument does not vary across ξ, i.e., the function is constant on all of ξ.

We desire to minimize (11) over $\mathcal{G}$. Repeating the same steps (3) to (10), a_0 should approximately satisfy the following:

$$\begin{aligned} M^T M a_0 = \mathbf{0}_N, \quad M &= \begin{pmatrix} \leftarrow & \sum_{0\le l<1}\left(g(t_l, x_l) - g(t_1, x_1)\right) & \rightarrow \\ & \vdots & \\ \leftarrow & \sum_{0\le l<n-1}\left(g(t_l, x_l) - g(t_{n-1}, x_{n-1})\right) & \rightarrow \end{pmatrix} \\ &= \begin{pmatrix} \leftarrow & \sum_{0\le l<1} g(t_l, x_l) - g(t_1, x_1) & \rightarrow \\ \leftarrow & \sum_{0\le l<2} g(t_l, x_l) - 2f(t_2, x_2) & \rightarrow \\ & \vdots & \\ \leftarrow & \sum_{0\le l<n-1} g(t_l, x_l) - (n-1)\, g(t_{n-1}, x_{n-1}) & \rightarrow \end{pmatrix}. \end{aligned} \tag{12}$$

3.3 The forward deviation seminorm

Similarly, when $b_{kl} = I_{l>k}$, we consider the functional

$$V(F) = \frac{1}{2}\sum_{k=0}^{n-2}\left[\sum_{k<l\leq n-1}\left(F(t_l,x_l) - F(t_k,x_k)\right)\right]^2, \tag{13}$$

and to find the minimum of (13) over $\mathcal{G}$, we can solve the equivalent problem

$$\begin{aligned} M^T M a_0 = \mathbf{0}_N, \quad M &= \begin{pmatrix} \leftarrow & \sum_{0<l\leq n-1}\left(g(t_l,x_l) - g(t_0,x_0)\right) & \rightarrow \\ & \vdots & \\ \leftarrow & \sum_{n-2<l\leq n-1}\left(g(t_l,x_l) - g(t_{n-2},x_{n-2})\right) & \rightarrow \end{pmatrix} \\ &= \begin{pmatrix} \leftarrow & \sum_{0<l\leq n-1} g(t_l,x_l) - (n-1)\,g(t_0,x_0) & \rightarrow \\ \leftarrow & \sum_{1<l\leq n-1} g(t_l,x_l) - (n-2)\,g(t_1,x_1) & \rightarrow \\ & \vdots & \\ \leftarrow & \sum_{n-2<l\leq n-1} g(t_l,x_l) - g(t_{n-2},x_{n-2}) & \rightarrow \end{pmatrix}. \end{aligned} \tag{14}$$

4 Method 1B: General sum-of-squares seminorms

We assume the same setup as in Section 3. Then, we can define another class of seminorms from $C(\Xi,\mathbb{R})$ to $\mathbb{R}_{\geq 0}$ as

$$W(F) = \frac{1}{2}\sum_{k=1}^{n-1}\sum_{0\leq l<k} b_{kl}\left(F(t_k,x_k) - F(t_l,x_l)\right)^2, \tag{15}$$

where $b_{kl} \in \mathbb{R}$ for all $(k,l) \in \{(x,y) : x \in \{1,\ldots,n-1\}, 0 \leq y < x\}$. Then, if we minimize (15) over $\mathcal{G}$, the best-fit vector of coefficients $a_0 \in \mathbb{R}^N$ will satisfy the matrix equation

$$K a_0 := \sum_{k=1}^{n-1}\sum_{0\leq l<k} b_{kl}\left(g(t_k,x_k) - g(t_l,x_l)\right)\left(g(t_k,x_k) - g(t_l,x_l)\right)^T a_0 = \mathbf{0}_N. \tag{16}$$

(16) gives us a matrix equation which we can solve for nontrivial solutions of a_0; all possible solutions for a_0 lie in $\ker K$. As in Section 3, in practice, ξ will exhibit some measurement error due to noise, and so K may only be near-singular. Hence, to reconstruct $\ker K$ and hence a_0, we can again appeal to the Eckart–Young–Mirsky theorem as we did in Section 3. By singular value decomposition, $K = Z\Gamma P^T$, where Z and P are orthogonal matrices with column vectors $z_1,\ldots,z_{n-1}$ and $p_1,\ldots,p_N$, respectively, and Γ is a diagonal matrix of the same size as K with entries $\gamma_1 \geq \cdots \geq \gamma_N$ that correspond to the singular values of K. Thus, by the Eckart-Young-Mirsky theorem, the best rank k approximation to K is $\sum_{j=1}^{k}\gamma_j z_j p_j^T$, such that $||Kp_j|| = \gamma_j$ for $j = 1,\ldots,N$ and we should therefore let $a_0 = p_N$. In fact, we could also let $a_0 = p_j$ for any $j \in \{1,\ldots,N-1\}$ such that $\gamma_j \leq \tau$ for some tolerance $0 \leq \tau \ll 1$, since (1) could exhibit multiple integrals of motion, and we may consider any linear combination of these candidate constants of motion as well too retrieve another candidate constant of motion.

TODO: Conduct error analysis: How does our solution for a_0 vary depending off of errors in K? See, e.g., https://arxiv.org/pdf/1801.10512.pdf and https://arxiv.org/pdf/1601.03678.pdf.

5 Method 2A: Phase volume integration (time-independent case)

In this case, suppose that we have a collection $\xi^{(1)}, \ldots, \xi^{(m)}$ of $m \in \mathbb{N}$ time series in our data (where $\bigcup_{i=1}^{m} \xi^{(i)} = \xi$), such that $\xi^{(k)} = \left\{x_0^{(k)}, \ldots, x_{n^{(k)}}^{(k)}\right\}$ is a trajectory of (1) associated with initial condition $x_0^{(k)}$ and containing $n^{(k)} \in \mathbb{N}$ points for $k = 1, \ldots, m$. In that case, for any integral of motion Q for (1), since Q is constant along trajectories that solve (1), we know that $Q\left(x_j^{(k)}\right) = C^{(k)}$ for any $j \in \left\{0, \ldots, n^{(k)} - 1\right\}$ and some constant $C^{(k)} \in \mathbb{R}$ for $k = 1, \ldots, m$. In this case, we want to determine the values $C^{(1)}, \ldots, C^{(m)}$ which Q could take along each trajectory through phase space, thereby determining the level sets of Q and prescribing the values of Q as a function.

By multivariable chain rule, $\nabla \cdot (Qf) = f \cdot \nabla Q + Q \nabla \cdot f$, and since $f \cdot \nabla Q = 0$ because Q is an integral of motion for (1), $\nabla \cdot (Qf) = Q \nabla \cdot f$ on all of O. Finally, integrating both sides over some compact sample volume $\Omega \subseteq O$ with piecewise smooth boundary $\partial\Omega$ and applying the divergence theorem, we conclude that

$$\int_{\Omega} Q \nabla \cdot f \, dV - \int_{\partial\Omega} Q f \cdot \widehat{n} \, dS = 0 \tag{17}$$

must hold for any such sample volume Ω. Then, if we populate our phase space O with a collection $\mathcal{W} = \{\Omega_1, \ldots, \Omega_M\}$ of compact subsets of $\mathbb{R}^d$ with piecewise smooth boundaries for some $M \in \mathbb{N}$ and integrate over each of these as in (17), we get a system of M integral equations which can be inverted for values of Q. Moreover, none of these integral equations involve derivatives of Q, which means that we can use the constants $C^{((1)}, \ldots, C^{(m)}$ exactly in (17) without using these to approximate any derivatives of Q numerically.

This phase volume formulation inspires us to come up with two reconstruction methods involving the general procedure above. However, both of these require us to know the vector field f which defines (1). f in some situations might be given. For instance, TODO: give some examples here. But nonetheless, there are algorithms in the literature, such as Brunton et al. (2016) (SINDy) and TODO: list some other methods here?, which give us a way to reconstruct f from the given time series data ξ when we do not know f a priori. Hence, we will assume from here on out that we know the values of f throughout O.

5.1 Basis expansion

As in Sections 3-4, suppose that $n_0 = 1$ such that (1) exhibits one integral of motion, Q, which we expand in the same finite basis of functions g such that $Q \approx \sum_{k=1}^{N} a_{0,k} g_k$ for a best-fit vector of coefficients a_0. Then, substituting this ansatz into (17),

$$\begin{aligned} &\int_{\Omega_j} \sum_{k=1}^{N} a_{0,k} g_k \nabla \cdot f \, dV - \int_{\partial\Omega_j} \sum_{k=1}^{N} a_{0,k} g_k f \cdot \widehat{n} \, dS \\ &= \sum_{k=1}^{N} a_{0,k} \left(\int_{\Omega_j} g_k \nabla \cdot f \, dV - \int_{\partial\Omega_j} g_k f \cdot \widehat{n} \, dS \right) \approx 0 \end{aligned} \tag{18}$$

for all $k = 1, \ldots, N$. Given $g = (g_1, \ldots, g_N)$ and f, the factors $\int_{\Omega_j} g_k \nabla \cdot f \, dV - \int_{\partial\Omega_j} g_k f \cdot \widehat{n} \, dS$ in (18) can be computed exactly or via numerical quadrature for all $(j, k) \in \{1, \ldots, M\} \times \{1, \ldots, N\}$, regardless of our choice of $\mathcal{W}$. Hence, (18) defines the following system of M

equations for N unknowns:

$$Ca_0 \approx \mathbf{0}_M,$$
$$C = \begin{pmatrix} \int_{\Omega_1} g_1 \nabla \cdot f \, dV - \int_{\partial\Omega_1} g_1 f \cdot \widehat{n} \, dS & \cdots & \int_{\Omega_1} g_N \nabla \cdot f \, dV - \int_{\partial\Omega_1} g_N f \cdot \widehat{n} \, dS \\ \vdots & \vdots & \vdots \\ \int_{\Omega_M} g_1 \nabla \cdot f \, dV - \int_{\partial\Omega_M} g_1 f \cdot \widehat{n} \, dS & \cdots & \int_{\Omega_M} g_N \nabla \cdot f \, dV - \int_{\partial\Omega_M} g_N f \cdot \widehat{n} \, dS \end{pmatrix}, \quad (19)$$

which we wish to invert approximately for some nontrivial a_0. Hence, we know that $a_0 \in \text{Null } C$ ideally, but due to errors in numerical quadrature and/or the computation of f, C might only be near-singular. As in Section 4, we can approximate C via singular value decomposition. Note that the basis expansion formulation does not directly require any knowledge of the time series data ξ itself, and hence, it also does not require knowing any information about which points correspond to which level sets of the given integral of motion Q. Hence, this makes such a method even suitable for finding (possibly approximate) integrals of motion in ODE systems where only the vector field f is known.

TODO: Conduct error analysis: How does our solution for a_0 vary depending off of errors in C?

5.2 Quadrature inversion

In a nutshell, the motivation behind quadrature inversion is straightforward for those who are familiar with numerical integration. For each sample volume Ω, we want to approximate the left side of (17) via numerical quadrature. By doing so, we obtain the linear equation

$$\sum_{j=1}^{o_1} b_j Q(x_j) (\nabla \cdot f)(x_j) - \sum_{j=1}^{o_2} c_j Q(y_j) f(y_j) \cdot \widehat{n}(y_j) = 0 \quad (20)$$

for some $o_1, o_2 \in \mathbb{R}$, along with quadrature weights and points $b_1, \ldots, b_{o_1}, c_1 \ldots, c_{o_2} \in \mathbb{R}$ and $x_1, \ldots, x_{o_1} \in \Omega, y_1, \ldots, y_{o_1} \in \partial\Omega$, respectively. The key here is that we enforce $Q(x_i) = Q(x_j) = C^{(k)}$ whenever $x_i, x_j \in \xi^{(k)}$ (or $Q(y_i) = Q(y_j) = C^{(k)}$ whenever $y_i, y_j \in \xi^{(k)}$) for the same $k \in \{1, \ldots, m\}$, such that x_i and x_j (respectively, y_i and y_j) belong to the same phase space trajectory that solves (1). Hence, it is preferable to choose our numerical quadrature method such that our quadrature points lie in ξ, because then we can use the time series data we already have to drastically reduce the dimensionality of our system by forcing Q to be constant along phase space trajectories. TODO: Is it necessary to impose that $Q(x_i) = Q(x_j) = C^{(k)}$ whenever $x_i, x_j \in \xi^{(k)}$ (or $Q(y_i) = Q(y_j) = C^{(k)}$ whenever $y_i, y_j \in \xi^{(k)}$)? Or can we leave this out? I am sure that it helps to enforce this when possible, but I wonder if the error analysis will reveal that either case still works.

Since we do not know the values $Q(x_j)$ or $Q(y_j)$ that correspond to level sets of our constant(s) of motion (else, the constant(s) of motion would already be determined), we want to set up M equations of the form (20) that correspond to numerical integrations over sample volumes $\Omega_1, \ldots, \Omega_M$, respectively, which we will then invert (perhaps approximately) for nontrivial values of $C^{(1)}, \ldots, C^{(m)}$ (or values of Q at arbitrary quadrature points if we do not use the data points in ξ) via the same SVD-derived method used in prior sections of this paper. Note that the less quadrature points we use in each equation of the form (20), the more sparse our resultant system of equations will be (and hence faster to invert), but with greater errors in numerical integration due to the use of lower-order quadrature.

5.3 Choosing sample volumes

An immediate question one might ask with either method outlined in Sections 5.1 and 5.2 above is the following: How do we choose the sizes, locations, and shapes of the sample volumes in $\mathcal{W}$? Moreover, which kind(s) of numerical quadrature are most suitable to use when computing volume and surface integrals over each sample volume in $\mathcal{W}$? Since the method outlined in Section 5.1 does not directly depend on ξ, we are under no particular inclination to choose $\mathcal{W}$ that follows any particular quadrature points, since we can compute the elements in C up to arbitrary precision, provided that we know f and $\nabla \cdot f$ everywhere in O. In that case, we can choose $\mathcal{W}$ completely arbitrarily, provided that $|\mathcal{W}| = M$ is sufficiently large and covers enough of O to provide us with enough information to compute a_0 accurately.

However, if f and $\nabla \cdot f$ are only known at certain points in O, then we cannot compute the integrals for each element of C exactly and must resort to numerical approximation. Similarly, for Section 5.2, we must also approximate the integral over each volume as a linear combination of values of Q at given points, preferably at points in ξ. In these cases, we have no control as to where these points are located a priori.

Suppose that f and $\nabla \cdot f$ are only known at points in a set X, and also let $\xi = X$ for notational simplicitly. We are inspired by the usage of d-dimensional Delaunay triangulation in space-discretised solvers such as the finite element method and the finite volume method to consider the choosing each $\Omega_j \in \mathcal{W}$ as follows: For each $x \in X$, assume that X is large and "spread out" enough such that we can find $d+1$ nearest neighbors of x by d-dimensional Euclidean distance which are in general position. Hence, these points comprise a set $S := \{z_0, z_1, \ldots, z_d\}$, where S is linearly independent. Then, S defines a d-simplex $\mathcal{S}$, i.e.,

$$\mathcal{S} := \left\{ \sum_{i=0}^{d} \lambda_i z_i : \sum_{i=0}^{d} \lambda_i = 1,\ \lambda_j \geq 0,\ j = 0, 1, \ldots, d \right\}, \tag{21}$$

with S comprising its vertices. ($\mathcal{S}$ is also known as the convex hull of S.) Then, each Ω_j can be associated with a point $x \in X$ that belongs to some nearest-neighbor simplex $\mathcal{S}$ as defined above, in which case $\Omega_j = \mathcal{S}$ and $M = \sum_{k=1}^{m} n^{(k)}$.

TODO: List or talk about some numerical algorithms which can be used to find nearest neighbors of this kind and construct simplices like this (of which there are many).

In both Sections 5.1 and 5.2, we are interested in applying numerical quadrature to integral equations of the form

$$\int_{\mathcal{S}} F \, dV - \int_{\partial \mathcal{S}} G \cdot \widehat{n} \, dS, \tag{22}$$

where we assume that $F \in C^2(\mathcal{S}, \mathbb{R})$ and $G \in C^2(\mathcal{S}, \mathbb{R})$, where $\mathcal{S}$ is a d-simplex. From Guessab and Schmeisser (2003) and Tommaso and Zerroudi (2020), there exist quadrature formulae for solving integrals of the forms in (22) which are second-order accurate (or higher). However, these all require knowing the values of F at the centroid of $\mathcal{S}$ and G at the centroids for each of the $d+1$ facets (i.e., the $(d-1)$-faces) of $\mathcal{S}$, since we can break up the second integral in (22) into a sum of integrals along each of the facets of $\mathcal{S}$, as these make up $\partial\mathcal{S}$. Some methods would also require knowing F at the centroids for the facets of $\mathcal{S}$, or G at the centroids for the facets of the facets of $\mathcal{S}$ (i.e., the $(d-2)$-faces of $\mathcal{S}$), but for sake of brevity, we will omit these quadrature formulae, since in the latter case, these would require us to invert many more matrices for linear interpolation across the $(d-2)$-faces of $\mathcal{S}$. To

that end, we rewrite (22) like

$$\int_{\mathcal{S}} F\, dV - \sum_{j=1}^{d+1} \int_{\mathcal{S}_j} G\left(x\right) \cdot \widehat{n}_j \, dS\left(x\right), \tag{23}$$

where $\mathcal{S}_j$ is the jth facet of $\mathcal{S}$ lying opposite to z_j for $j = 0, 1, \ldots, d$, and $\widehat{n}_j$ is the unit normal to $\mathcal{S}_j$, constant over all $x \in \mathcal{S}_j$ because $\mathcal{S}_j$ is a facet. Hence, we note that $\mathcal{S}_j$ has vertices $z_0, \ldots, z_{j-1}, z_{j+1}, \ldots, z_d$.

A natural local coordinate system on a simplex is the barycentric coordinate system, which expresses each element in $\mathcal{S}$ as a linear combination of the same $\lambda_0, \lambda_1, \ldots, \lambda_d$ used in (21) for defining $\mathcal{S}$, each of which is a function of the point in question. Accordingly, to solve for $\lambda_0^*, \lambda_1^*, \ldots, \lambda_d^*$, which are the barycentric coordinates for the centroid $x^* = \frac{1}{d+1} \sum_{i=0}^{d} z_i$ of $\mathcal{S}$, we note that $\sum_{i=0}^{d} \lambda_i^* = 1 \Rightarrow \lambda_0^* = 1 - \sum_{i=1}^{d} \lambda_i^*$, and we want to choose $\lambda_1^*, \ldots, \lambda_d^*$ such that

$$x^* = \lambda_0^* z_0 + \sum_{i=1}^{d} \lambda_i^* z_i = \left(1 - \sum_{i=1}^{d} \lambda_i^*\right) z_0 + \sum_{i=1}^{d} \lambda_i^* z_i \Rightarrow \sum_{i=1}^{d} \lambda_i^* \left(z_i - z_0\right) = x^* - z_0. \tag{24}$$

(24) defines a $d \times d$ system of linear equations which can be uniquely inverted for $\lambda_1^*, \ldots, \lambda_d^*$, provided that S is linearly independent (which we are assuming is the case here). Hence, a useful linear approximation for $F\left(x^*\right)$ is simply $F\left(x^*\right) \approx \left(1 - \sum_{i=1}^{d} \lambda_i^*\right) F\left(z_0\right) + \sum_{i=1}^{d} \lambda_i^* F\left(z_i\right) = F\left(z_0\right) + \sum_{i=1}^{d} \lambda_i^* \left(F\left(z_i\right) - F\left(z_0\right)\right)$. Similarly, for $\mathcal{S}_j$, if $\lambda_{j,0}^*, \ldots, \lambda_{j,j-1}^*, \lambda_{j,j+1}^*, \ldots, \lambda_{j,d}^*$ are the barycentric coordinates for the centroid x_j^* of $\mathcal{S}_j$ (and we can solve for these using the same procedure outlined above), then our linear approximation for $G\left(x_j^*\right) \cdot \widehat{n}_j$ at the centroid is just $G\left(x_0^*\right) \cdot \widehat{n}_0 \approx \left[G\left(z_1\right) + \sum_{i=2}^{d} \lambda_{0,i}^* \left(G\left(z_i\right) - G\left(z_1\right)\right)\right] \cdot \widehat{n}_0$ and $G\left(x_j^*\right) \cdot \widehat{n}_j \approx \left[G\left(z_0\right) + \sum_{\substack{i=1 \\ i \neq j}}^{d} \lambda_{j,i}^* \left(G\left(z_i\right) - G\left(z_0\right)\right)\right] \cdot \widehat{n}_j$ for $j = 1, \ldots, d$.

In either case, using the linear approximation for evaluating the function at the centroid introduces a second-order error term. But as derived in Guessab and Schmeisser (2003) and Tommaso and Zerroudi (2020), there exist a family of numerical formulae for any $Q \in C^2\left(\mathcal{S}, \mathbb{R}\right)$ that only require knowing Q at the vertices of $\mathcal{S}$ and at the centroid for $\mathcal{S}$. These are the following, which hold for any $\alpha \in [0, 1]$:

$$\frac{1}{|\mathcal{S}|} \int_{\mathcal{S}} Q\left(x\right)\, dx = \alpha Q\left(x^*\right) + \frac{1 - \alpha}{1 + d} \sum_{i=0}^{d} Q\left(z_i\right) + R_d^*\left(\alpha\right)\left[Q\right], \tag{25}$$

where $R_d^*\left(\alpha\right)\left[Q\right] \lesssim h^2$ and $h = \max_{i=0,1,\ldots,d} ||z_i - x^*||$. In fact, when $\alpha = \frac{d+1}{d+2}$, $R_d^*\left(\alpha\right)\left[Q\right] \lesssim h^3$, in which case (25) becomes the multivariate Simpson's rule. Another formula is the multivariate Hammer formula, which we get when we let $\alpha = \frac{1}{d+1}$ in (25). Either way, all such numerical integration formulae are at least second-order, and thus, using linear interpolation in barycentric coordinates to approximate the function value at each centroid will not change the order of the method.

Using (25) and noting the linear approximation for evaluating integrands at the centroids,

(23) becomes

$$
\begin{aligned}
&|\mathcal{S}|\left[\left(\frac{1+\alpha d}{1+d}-\alpha\sum_{i=1}^{d}\lambda_i^*\right)F\left(z_0\right)+\sum_{i=1}^{d}\left(\frac{1-\alpha}{1+d}+\alpha\lambda_i^*\right)F\left(z_i\right)\right]\\
&-|\mathcal{S}_0|\left[\left(\frac{1+\alpha_0\left(d-1\right)}{d}-\alpha_0\sum_{i=2}^{d}\lambda_{0,i}^*\right)G\left(z_1\right)+\sum_{i=2}^{d}\left(\frac{1-\alpha_0}{d}+\alpha_0\lambda_{0,i}^*\right)G\left(z_i\right)\right]\cdot\widehat{n}_0\\
&-\sum_{j=1}^{d}|\mathcal{S}_j|\left[\left(\frac{1+\alpha_j\left(d-1\right)}{d}-\alpha_j\sum_{\substack{i=1\\i\neq j}}^{d}\lambda_{j,i}^*\right)G\left(z_0\right)+\sum_{\substack{i=1\\i\neq j}}^{d}\left(\frac{1-\alpha_j}{d}+\alpha_j\lambda_{j,i}^*\right)G\left(z_i\right)\right]\cdot\widehat{n}_j+\mathcal{O}\left(h^2\right)
\end{aligned}
\tag{26}
$$

for *any* suitable choices of $\alpha,\alpha_0,\ldots,\alpha_d\in[0,1]$. As a further note, we could have also integrated the linear barycentric interpolation formulae for F and G explicitly over $\mathcal{S},\mathcal{S}_1,\ldots,\mathcal{S}_{d+1}$ using the magic formula (see Vermolen and Segal (2018) or Cuvelier (2018)), but it turns out that doing so gives the same result as (26) for $\alpha=0$.

If inverting the system of equations in (24) is too expensive (especially when computing the surface integral over $\partial\mathcal{S}$, which would require $d+1$ such inversions), we can let $\alpha=\alpha_0=\cdots=\alpha_d=0$. In this case, the term with $Q\left(x^*\right)$ disappears entirely from (25) and we do not need to calculate λ^* and the λ_j^*'s in (26), which saves us from inverting a $d\times d$ matrix and $d\left(d-1\right)\times\left(d-1\right)$ matrices. Evidently, the most expensive operation is computing the λ_j^*'s; when using usual matrix inversion algorithms, the time complexity will be $\mathcal{O}\left(d^4\right)$, while computing λ^* is only $\mathcal{O}\left(d^3\right)$. Hence, for the reader's convenience (and because this is the formula we will use in simulations), (26) becomes the following when $\alpha_0=\cdots=\alpha_d=0$ (we still leave α up to the reader's choice):

$$
\begin{aligned}
&|\mathcal{S}|\left[\left(\frac{1+\alpha d}{1+d}-\alpha\sum_{i=1}^{d}\lambda_i^*\right)F\left(z_0\right)+\sum_{i=1}^{d}\left(\frac{1-\alpha}{1+d}+\alpha\lambda_i^*\right)F\left(z_i\right)\right]\\
&\qquad-\frac{1}{d}\sum_{j=0}^{d}|\mathcal{S}_j|\,\widehat{n}_j\cdot\sum_{\substack{i=0\\i\neq j}}^{d}G\left(z_i\right)+\mathcal{O}\left(h^2\right).
\end{aligned}
\tag{27}
$$

For Section 5.1, we set $F=g_k\nabla\cdot f$ and $G=g_kf$ for $k=1,\ldots,N$, while for Section 5.2, we need $F=Q\nabla\cdot f$ and $G=Qf$, and in both cases, $\mathcal{S}=\Omega_l$ for $l=1,\ldots,M$. The main difference is that, for Section 5.1, since F and G are known at the query points given, (26) (or (27)) gives us each entry of C directly, while for Section 5.2, we set (26) (or (27)) equal to zero and write Q in terms of $C^{(1)},\ldots,C^{(m)}$, depending on the given query points, for $\mathcal{S}=\Omega_l$ and each $l=1,\ldots,M$, thereby setting up a system of M equations for m unknowns which can be (approximately) inverted for $C^{(1)},\ldots,C^{(m)}$ via SVD and the methods given in Section 3.

5.3.1 Simplex formulae

We have yet to give formulae for computing $|\mathcal{S}|$, $|\mathcal{S}_j|$, and $\widehat{n}_j$, which are the volume of $\mathcal{S}$, the volume of $\mathcal{S}_j$, and the unit normal to $\mathcal{S}_j$, respectively. The volume of an N-simplex with its vertices consisting of N points is given by a formula involving the Cayley-Menger Determinant (https://mathworld.wolfram.com/Cayley-MengerDeterminant.html). In

particular, for $|\mathcal{S}|$, we have that

$$|\mathcal{S}|^2 = \frac{(-1)^{d+1}}{2^d (d!)^2} \det A = \left| \frac{1}{d!} \det E \right|, \tag{28}$$

where $A \in \mathbb{R}^{(d+2)\times(d+2)}$ such that for all $(i,k) \in \{1,\ldots,d+2\}^2$,

$$[A]_{ik} = \begin{cases} ||z_{i-1} - z_{k-1}||_2^2 & i,k = 1,\ldots,d+1, \\ 0 & i = k, \\ 1 & \text{else}, \end{cases} \tag{29}$$

and $E \in \mathbb{R}^{(d+1)\times(d+1)}$ where

$$E = \begin{pmatrix} z_0^T & 1 \\ z_1^T & 1 \\ \vdots & \vdots \\ z_d^T & 1 \end{pmatrix}. \tag{30}$$

One can also compute similar formulae for $|\mathcal{S}_0|, |\mathcal{S}_1|, \ldots, |\mathcal{S}_d|$ by analogy with (28)-(30).

To find the unit normal $\widehat{n}_j$ to $\mathcal{S}_j$, consider the inverse of E, which can be expressed in the form below:

$$E^{-1} = \begin{pmatrix} n_0 & n_1 & \ldots & n_d \\ c_0 & c_1 & \ldots & c_d \end{pmatrix}, \tag{31}$$

where $c_0, c_1, \ldots, c_d \in \mathbb{R}$ such that $\langle z_i, n_j \rangle = \delta_{ij}(1 + c_j - c_i) - c_j$ for $(i,j) \in \{0,\ldots,d\}^2$. TODO: Explain/show why E is invertible iff the points are in general position and has the form given below. Hence, it is not hard to show from these properties that n_j is the (possibly non-normalized) normal vector to S_j for $j = 0,1,\ldots,d$. Then, $\widehat{n}_j = n_j / ||n_j||_2$ for $j = 0,1,\ldots,d$.

5.3.2 Working with degenerate or low-information cases

If there is some $x \in X$ such that we cannot find d nearest neighbors satisfying the condition above, or if forcing these nearest neighbors and x to be linearly independent leads to too many redundant sample volumes, then a sufficiently large subset of X lies in some lower-dimensional subspace S of $\mathbb{R}^d$, such that $\dim S = s < d$, or X is not large enough, i.e., $n < d+1$. But in the latter case, we can still possibly extract useful information from the system by considering a suitable lower-dimensional subspace S with $s = n-1$. Either way, we consider the projection of our vector field f and data X onto S, which should take the form of some suitable linear transformation. TODO: Attach some sources here and maybe describe what this should look like explicitly. Then, for each $x \in S$, we can find s nearest neighbors of x by s-dimensional Euclidean distance on S, where x and these s nearest neighbors are linearly independent. Then, we can apply the same techniques featured in Section 5.3, except this time, we have a phase space integral over an s-simplex with x and the s nearest neighbors of x as its vertices. And of course, if some point in S does not have s nearest neighbors satisfying this condition, then we can continue the process iteratively until we either have enough sample volumes to suitably reconstruct $C^{(1)}, \ldots, C^{(m)}$ or we have exhausted every $x \in X$

TODO: Extend these results for when the time series data lies on some lower-dimensional manifold? In that case, it would be necessary to do nonlinear dimensionality reduction and project the system onto this lower-dimensional manifold.

6 Method 2B: Phase volume integration (time-dependent case)

We start with the same setup as for Method 2A, except in this case, Q is a *time-dependent* constant of motion for (1), such that $Q\left(t_j^{(k)}, x_j^{(k)}\right) = C^{(k)}$ for any $j \in \left\{0, 1, \ldots, n^{(k)} - 1\right\}$ and some constant $C^{(k)} \in \mathbb{R}$ for $k = 1, \ldots, m$. As before, we want to determine $C^{(1)}, \ldots, C^{(m)}$ which Q takes along each trajectory through phase space, and since Q is time-dependent, $f \cdot \nabla Q = -\partial_t Q$. We also populate our phase space O with the same collection $\mathcal{W}$ of compact subsets of $\mathbb{R}^d$ with piecewise smooth boundaries. Then, by multivariable chain rule, $\nabla \cdot (Qf) = Q\nabla \cdot f - \partial_t Q$ on all of O, and after integrating both sides over some $\Omega_j \in \mathcal{W}$ and applying the divergence theorem, we get that

$$\int_{\Omega_j} \frac{\partial Q}{\partial t}\, dV = \int_{\Omega_j} Q\nabla \cdot f\, dV - \int_{\partial\Omega_j} Qf \cdot \widehat{n}\, dS \tag{32}$$

must hold for any such $\Omega_j \in \mathcal{W}$, and like with (17), we see that (32) does not require any spatial derivatives of Q. As before, we assume that we know f a priori on O.

6.1 Basis expansion

Like in the above sections, let $n_0 = 1$ and expand Q in a finite basis of functions g such that $Q \approx \sum_{k=1}^{N} a_{0,k} g_k$ for a vector of best-fit coefficients a_0. Then, substituting this ansatz into (32),

$$\int_{\Omega_j} \frac{\partial}{\partial t} \sum_{k=1}^{N} a_{0,k} g_k\, dV - \int_{\Omega_j} \sum_{k=1}^{N} a_{0,k} g_k \nabla \cdot f\, dV + \int_{\partial\Omega_j} \sum_{k=1}^{N} a_{0,k} g_k f \cdot \widehat{n}\, dS \tag{33}$$

$$= \sum_{k=1}^{N} a_{0,k} \left[\int_{\Omega_j} \left(\frac{\partial g_k}{\partial t} - g_k \nabla \cdot f \right) dV + \int_{\partial\Omega_j} g_k f \cdot \widehat{n}\, dS \right] \approx 0 \tag{34}$$

for all $k = 1, \ldots, N$. Given $g = (g_1, \ldots, g_N)$ and f, the factors $\int_{\Omega_j} (\partial_t g_k - g_k \nabla \cdot f\,)\, dV + \int_{\partial\Omega_j} g_k f \cdot \widehat{n}\, dS$ in (34) can be computed exactly or via numerical quadrature for all $(j, k) \in \{1, \ldots, M\} \times \{1, \ldots, N\}$ for arbitrary $\mathcal{W}$. Hence, (18) defines the following system of M equations for N unknowns:

$$Ca_0 \approx \mathbf{0}_M, \quad [C]_{jk} = \int_{\Omega_j} \left(\frac{\partial g_k}{\partial t} - g_k \nabla \cdot f \right) dV + \int_{\partial\Omega_j} g_k f \cdot \widehat{n}\, dS,$$
$$(j, k) \in \{1, \ldots, M\} \times \{1, \ldots, N\}, \tag{35}$$

which we can invert approximately for some nontrivial a_0 in the same manner as in Section 5.1.

TODO: Conduct error analysis: How does our solution for a_0 vary depending off of errors in C?

6.2 Matrix exponential method

Any constant of motion $Q : [t_0, t_{n-1}] \times O \to \mathbb{R}$ for (1) should satisfy the PDE

$$\frac{\partial Q}{\partial t} = (\nabla \cdot f)\, Q - \nabla \cdot (fQ)\,. \tag{36}$$

We discretize Q into a grid function defined in connection with the collection of phase space volumes $\mathcal{W}$ from Section 5 — namely, we represent Q as a grid function $\widehat{Q}$ with M values at $\widehat{x} \in \mathbb{R}^{d \times M}$. After integrating both sides of (36) over Ω_j and applying the divergence theorem,

$$\int_{\Omega_j} \frac{\partial Q}{\partial t}\, dV = \int_{\Omega_j} \left[(\nabla \cdot f)\, Q - \nabla \cdot (fQ) \right] dV = \int_{\Omega_j} (\nabla \cdot f)\, Q\, dV - \int_{\partial \Omega_j} (f \cdot \widehat{n})\, Q\, dS. \tag{37}$$

We can approximate both sides of (37) via numerical quadrature methods with quadrature points chosen on the grid upon which we defined $\widehat{Q}$. In particular, if we use methods that are order p, we obtain the linear equation

$$A_1 \frac{d\widehat{Q}}{dt} = A_2 \widehat{Q} - A_3 \widehat{Q} = (A_2 - A_3)\, \widehat{Q} + \mathcal{O}\left(d \max_j |\Omega_j|^p \right) + \mathcal{O}\left(d \max_j |\partial \Omega_j|^p \right) \tag{38}$$

for quadrature weights $A_1, A_2, A_3 \in \mathbb{R}^{M \times M}$ which are used to approximate the integral operators in (37) as follows:

$$\begin{aligned}
A_1 \frac{d\widehat{Q}}{dt} &= \begin{pmatrix} \int_{\Omega_1} \frac{\partial Q}{\partial t}\, dV \\ \int_{\Omega_2} \frac{\partial Q}{\partial t}\, dV \\ \vdots \\ \int_{\Omega_M} \frac{\partial Q}{\partial t}\, dV \end{pmatrix} + \mathcal{O}\left(d \max_j |\Omega_j|^p \right) + \mathcal{O}\left(d \max_j |\partial \Omega_j|^p \right), \\
A_2 \widehat{Q} &= \begin{pmatrix} \int_{\Omega_1} (\nabla \cdot f)\, Q\, dV \\ \int_{\Omega_2} (\nabla \cdot f)\, Q\, dV \\ \vdots \\ \int_{\Omega_M} (\nabla \cdot f)\, Q\, dV \end{pmatrix} + \mathcal{O}\left(d \max_j |\Omega_j|^p \right) + \mathcal{O}\left(d \max_j |\partial \Omega_j|^p \right), \\
A_3 \widehat{Q} &= \begin{pmatrix} \int_{\partial \Omega_1} (f \cdot \widehat{n})\, Q\, dS \\ \int_{\partial \Omega_2} (f \cdot \widehat{n})\, Q\, dS \\ \vdots \\ \int_{\partial \Omega_M} (f \cdot \widehat{n})\, Q\, dS \end{pmatrix} + \mathcal{O}\left(d \max_j |\Omega_j|^p \right) + \mathcal{O}\left(d \max_j |\partial \Omega_j|^p \right).
\end{aligned} \tag{39}$$

Hence, it might be valuable to choose $\widehat{x}$ such that each volume Ω_j contains a sufficiently large number of grid points within and around it, as this would thereby lower the discretization error introduced via numerical quadrature.

Assuming that A_1 is invertible — which should be the case for uniquely chosen volumes Ω_j — we can use (38) to solve for $\frac{\partial \widehat{Q}}{\partial t}$, such that

$$\frac{d\widehat{Q}}{dt} = A_1^{-1} (A_2 - A_3)\, \widehat{Q} + \mathcal{O}\left(d \max_j |\Omega_j|^p \right) + \mathcal{O}\left(d \max_j |\partial \Omega_j|^p \right), \quad \widehat{Q}(0) = \widehat{Q}_0 \tag{40}$$

for some initial condition $\widehat{Q}_0 \in \mathbb{R}^M$ to be determined. Since the vector field f for (1) is autonomous, A_1, A_2, and A_3 do not depend off of time, and hence, (40) can be solved approximately yet explicitly for all times $t > 0$ using the matrix exponential: $\widehat{Q}(t) = \exp\left(A_1^{-1} (A_2 - A_3)\, t \right) \widehat{Q}_0$. This solution is not without a spatial discretization error, TODO: Analyze how the discretization error affects the solution.

However, how do we find Q_0? Before we do so, we give some guiding principles. A constant of motion $Q(t, x(t))$ reflected in the trajectory $x(t)$ that solves (1) for a given initial condition $x(0) = x_0$ has its own initial condition $Q(0, x_0) = Q_0$, and $Q(t, x(t))$

remains equal to Q_0 for all time. If we consider the set of all possible initial conditions, we can track the time evolution of Q as a codimension-one surface in $\mathbb{R}^{d+1}$. Naturally, since Q is a well-defined, single-valued function from $\mathbb{R}^{d+1}$ to $\mathbb{R}$, there will be a unique initial condition $Q(0,x) = Q_0(x)$ corresponding to the correct constant of motion Q that both satisfies $Q = \exp\left(A_1^{-1}(A_2 - A_3)t\right)\widehat{Q}_0$ and is actually a constant of motion for (1).

Since $\widehat{Q}_0$ gives the coefficients for the optimal linear combination of column vectors for $\exp\left(A_1^{-1}(A_2 - A_3)t\right)$, the columns of $\exp\left(A_1^{-1}(A_2 - A_3)t\right)$ form a finite basis of $M \in \mathbb{N}$ grid functions. By interpolation, we can extend this column basis to a basis of functions from Ξ to $\mathbb{R}$, thereby allowing us to use any of the methods from Sections 3 and 4 when we let $M = N$ and $g_j = \left[\exp\left(A_1^{-1}(A_2 - A_3)t\right)\right]_{(\cdot)j}$ for $j = 1, \ldots, N$, in which case $\widehat{Q}_0 = a_0$. Moreover, we can also use the column space of $\exp\left(A_1^{-1}(A_2 - A_3)t\right)$ to supplement a finite basis of functions which we might already have — this corresponds to when $M < N$. Finally, if $n = M$, then we will not actually need to interpolate each column of $\exp\left(A_1^{-1}(A_2 - A_3)t\right)$ since we can just choose our spatial grid to coincide with our time series, i.e., we can let $\widehat{x} = \xi$.

Most of the reconstruction methods involve us having to find the (perhaps approximate) solution to a system of linear equations. In all cases, this solution lies in some vector space of functions (e.g., $\mathcal{L}^p\left(\mathbb{R}^d\right)$) and is represented by a best-fit linear combination of basis functions in that space. If we want to talk about how good of a fit this linear combination is, or if the solution space we find is close to those spanned by the actual constants of motion exhibited by (1), then we must equip this vector space with a norm to measure convergence and "closeness" of functions. Finally, finding the solution in an infinite-dimensional vector space of functions is generally very difficult to do, but we can use some finite-dimensional function space which is dense or can approximate this space to an acceptable degree.

Keeping these motivations in mind, we prove several theorems and properties which relate the distance of a vector to the solution set of a system of linear equations over (possibly infinite-dimensional) normed vector spaces to the distance of their images in $\mathbb{R}$. This is done in another document, "Error analysis for linear systems over general normed spaces."